

git config --local user.name/email " <name/email> ": identify myself. --global set default for all repos.
git clone <repo-url> [<dir/empty>]
remote add <name> <url>; remote show <name>; ls-remote
git status: show current branch, state of files(staged/modified&unstaged /untracked), suggest next git cmd.
git log: shows commit history of current branch. -n: only n commits. -p: each commit diff (line add/removed).
git diff: unstaged changes vs working tree. --staged staged vs last commit. --check: whitespace errors.
git reflog: show local history of where HEAD has pointed. **branch rescue <reflog-hash>:** find lost commit
git commit; --amend: edit previous commit msg without alter snapshot. -m: prevent open vim editor to write log msg. -a: skips 'git add' for tracked files only.
git restore <file>: discard unstaged edits in working dir, overwrite by <file> from index. If nothing has been staged, index ≈ last commit (HEAD). **git reset <file>:** index reset to match HEAD; working dir unchanged.
git reset mode <commit>: mv reference pointers (i.e. HEAD). HEAD~N=N-th ancestor of current commit.
--mixed (default): mv HEAD to <commit>; index reset to that commit, working dir unchanged(changes become unstaged). --soft: mv HEAD; index+working dir keep changes(staging area remain). --hard: HEAD, index, working dir all reset to <commit>.
git revert <commit>: create a new commit that applies the inverse of <commit>. safe (no rewrite history).
git branch: list branches; shows current branch.
git branch <name>; git checkout <name>: move HEAD.
• **git checkout -b <name> origin/<name>:** create local tracking branch from remote branch.
git merge <commit-or-branch>
git rebase <base>: replay current branch's commits on top of <base> (new commit IDs; linear history).
git fetch <remote> <branch>: update remote tracking branch (e.g. origin/master) without merging.
git pull: fetch+merge; **git pull --rebase:** fetch+rebase local commits on top of updated remote, no extra merg.
git stash: stash tracked changes.
stash save "msg": with a msg. **stash list:** list all stashes.
show <stashID>: ID's summary. **stash pop:** apply latest stash and remove it from stash list.
git clean: -n: dry run. -f: remove untracked files
git fetch <remote> <branch>: sync branch from remote.
Macro:checkout master, fetch blessed master, merge blessed/master or rebase blessed/master.
New feature: stash, clean -n/-f, **Macro**, solve conflict, checkout -b fe/f2 +merge f2 to master: add+commit, **Mac**, merge fe/f2, push blessed master, branch -d fe/f2.
Hotfix in master: **Mac**, checkout -b hotfix, edit+commit, **Mac**, merge hotfix, push blessed master, branch -d hotfix.
Make Syntax: if no dependency and target exist→cmd never run. if run "make" with no specified task, it will run the first non-pattern-ruled target in the file, and stop.
\$(var): a variable var; @\$=target name %<class>: %<java>
\$<=first dependency; ^<=all dependencies javac <\$<
run make Hello.class→to build Hello.class, use Hello.java

When defects introduced: most injected -coding; too found-system test; bugs find later-more expensive.
Waterfall Model (traditional, plan-driven): Linear, phase-by-phase (requirements→design→code→test → deploy). **Pros:** Easy to manage, clear milestones.
Cons: Intensive planning, Hard to adapt user needs.
V-Model (Validation & Verification model): Extension of Waterfall→each development phase has a matching testing phase. To catch defects early.
Spiral Model: Iterative & risk-driven--each loop adds functionality. **Each cycle:** planning→risk analysis→engineering (code and test)→evaluation (review result and plan for next). **Pros:** Good for large, risky projects (e.g., spacecraft control systems). **Cons:** repeated risk analysis→expensive, need skillful team.
Rational Unified Process (RUP): A heavyweight, iterative, incremental model used by large enterprises (IBM). plan-driven, process-intensive. **Phases:** Inception→Elaboration→Construction→Transition. Emphasizes comprehensive documentation, architecture-first approaches, defined roles & artifacts
Agile Model: What: lightweight, iterative, customer-focused; **Pros:** suitable for projects with rapidly changing requirements & a need for quicker delivery.
Heavy weight: plan-driven; huge SRS; change is expensive. use for large predictable projects.
Light weight = agile: planning is incremental & continual; easier to change. use for fast-changing and uncertain projects.
Artifacts = the tangible outputs of development (requirements, code, docs). **Evolution:** All artifacts go through continuous changes and need management.
RCS (Branching support): local, file-by-file history, introduced real branching; good for single developer workflow. Pessimistic concurrency model (lock-modify-unlock)→didn't support teams. **Centralized (CVS/SVN):** Optimistic concurrency model (copy-modify-merge): non-conflicting edits merge together automatically. easy control of history. Single point of failure; Network dependency; Awkward branching & heavy merging. **Distributed (Git/Mercurial):** every clone is full repo & history; local commits [offline]; eventual consistency. Complex management of history; more disk memory (full history locally).
Delta-based VCS (C/S/M): One full set of files and a chain of per-file changes (deltas) over time. Reconstruct a past version by applying the deltas.
Snapshot-based VCS (G): Each commit is a snapshot of the entire project (blobs). Unchanged files are just pointers to previously stored content.
Remote references & Remote-tracking branches: A reference (ref) = "a named pointer to a commit"
Remote references are refs that actually live on the remote repository (branches, tags, etc. on GitHub). Remote-tracking branches (like origin/master) are your local read-only copies of some remote branch.
Tracked files: appear in commits or staging area
Untracked files: in working dir and gitignore

Git 3 places: Working directory, Staging area(binary file for what will be in next commit), Repository
Git 3 states: Modified→Staged→Committed
Git commits is in linked-list: Branches are movable pointers, creating/switching is cheap; HEAD tracks current branch. Fast-forward merges when linear; Three-way merge when histories diverge.
Integration-Manager model (simplest): **1.** The project maintainer pushes to their public repository. **2.** A contributor clones that repository and makes changes. **3.** The contributor pushes to their own public copy. **4.** The contributor sends the maintainer an email asking them to pull changes. **5.** The maintainer adds the contributor's repository as a remote and merges locally **6.** The maintainer pushes merged changes to the main (blessed) repository.
Dictator & Lieutenants (Used by projects with hundreds of devs): **1.** Developers work on their topic branch and rebase their work on top of master. **2.** The master branch is that of the reference repository to which the dictator pushes. **3.** Lieutenants merge the developers' topic branches into their master branch. **4.** Dictator merges lieutenants' master branches into dictator's master branch. **5.** Dictator pushes that master branch to the reference repository→other developers can rebase on it.
Configuration Management(CM) 4 key activities: System building: Assemble code, config file, data file, libraries, and compiler into executable.
Version -: track component versions, solve conflict.
Change -: manages changes, cost, implem decision.
Release management: track/prepare external release.
Version Management(VM)=the process of managing
CodeLine: a sequence of versions of source code &
Baseline: a definition of a specific system (config file)
Semantic Versioning: Given a version number MAJOR.MINOR.PATCH, increment: MAJOR: incompatible API changes. MINOR: add functionality in a backwards-compatible manner. PATCH: make backwards-compatible bug fixes.
Software Build Automation Tools(4)
idempotent: result the same when run multiple times.
Make (Oldest, default on most systems): make execute command when the target is older the the dependency or the target doesn't exist. **Apache Ant** (XML, no convention): Too flexible. need to specify too many things. no imposed dependency management framework. **Apache Maven** (XML, convention over configuration →reduce config needs): **Features central repository for depend- management with transitive dependencies. Rigid structure requirements, verbose XML. Gradle** (use Groovy DSL) : **Combines Ant's flexibility with Maven's conventions. Task-based. Manage full build LC:** Initialization (identify projects in the build)→Configuration (tasks assembled into DAG. set up for task actions)→Execution(run tasks in order)
1.Ppl&cmu solve problems; process/tools only support;
2.Running code proves value; docs just describe it.
3.Build with the customer, not just to a contract.

Testing Principles: Design for Testability: software isn't designed with testing in mind→harder&expensive to test effectively. **Keep Tests Independent:** tests depend on each other→failures cascade, difficult to identify actual problems. **Verify One Condition per Test:** makes tests easier to understand/debug/maintain. **Write the Tests First: TDD.** Communicate Intent: Tests should be easy understand. **Don't Modify the SUT(System Under Test):** Avoid modifying parts of code that we are actually testing for. **Minimize Untestable Code:** Avoid writing code that is hard to automatically test.
Software Test Modeling. Defective(Negative) Testing→ensure robustness. **Validation(Positive) Testing→**builds confidence. **RIPR model of faults:** software as a simple input-output system during testing; reached, induced, propagate, revealed.
Types of Defects. Syntax, Runtime, Logic, Timing Error.
Software Testing Levels (What, Why). Functional (Unit) Testing: Tests individual components in isolation.
Integration Testing: Tests interactions between components. **System Testing:** Tests the functionality of whole program. **Acceptance Testing:** Tests the system from user perspective. Verifies that the system meets business requirements and is ready for deployment.
Regression Testing (Not a level): The process of re-running existing tests after changes (new features/bug fixes) to ensure previously working feature still works.
Equivalence partitioning: groups inputs by common behavior. **Boundary-value testing:** focuses on the edge cases where different behaviors meet. **They** are often used together, with BVT being a refinement of EP.
Main categories of Testing Smell (& examples)
Code Smells: Coding-level anti-patterns noticed while reading/writing test code (affect quality&maintainability of test code itself). **Behavior Smells:** Affect test outcomes during execution (compile errors, test failures). E.g., Fragile test (interface, behavior, data, context sensitivity). **Project Smells:** Indicators of overall project health, noticed by managers/customers. E.g. Production bugs (higher than expected defects found in formal testing), Buggy Tests, High Test Maintenance Cost (tests requiring modification too frequently), Developers Not Writing Tests.
Test Double Patterns (What). Dummy: Objects passed around but never used. Used to fill parameter lists.
Stubs: Provides canned answers to calls. Returns predetermined responses for testing. (e.g., create a function that always return false). **Mock:** Verifies interactions. (e.g, check if another function received a call from our testing function). **Fake:** Simpler than real implementations (e.g., in-memory database). **Spy:** Records how it was used for later verification. (e.g., does it change value in another class? we can create local simplified class to check changed field)
Coverage Criteria: Method, Statement, Branch, Condition, Loop.

Author: Wenyu (Wendy) Fan

4.Embrace change to avoid building obsolete features.

CI (before a merge): on every commit to mainline, CI server runs automated build+test on a clean workspace. fail→highest priority→fix.
CD (after): packages a versioned artifact, runs it (automated deploy pipel), keep it in a releasable state.
Continuous Delivery: manually deploy at any time,
Continuous Deployment: automatically deploy
CI workflow steps:

1. Developer checks out mainline from VCS
2. Build+run tests locally: if fail, fix first.
3. Developer makes changes (e.g., add feature), commit
4. Build+rerun tests locally: if fail, fix first; if pass, commit to build system server.
5. Build+test on build server: merge changes committed by other developers since the last commit, and tests should pass on developer's workspace
6. If conflicts with others' commits, then fix locally
7. Commit final changes to mainline

Effective CI Automation Practice: Regular Check-in (small, frequent commits); Comprehensive automated tests-Unit (small piece in isolation), Component (behavior of several components), Acceptance testing (meet acceptance criteria of end users, func&non-func); Keep Build/Test Short (1.5-10mins); Dev env≈CI env; Use CI software.

Effective CI Team Practice:

- Don't check in on a broken build→don't hide cause
- Run tests locally or let CI server do it.
- Wait for commit tests to pass→ catch probs early.
- Never go home on a broken build→mainl releasable.
- Always be prepared to revert ("airplane pilot").
- Time-Box fixing (If not fixed within X mins→revert).
- Do Not comment out failing tests→reliable safety.
- Take responsibility for all breakages in your changes

Jenkins = OSS automation server supports CI/CD (stages:build/test/deploy), JUnit report integration, artifact storage, and fast feedback.

Jenkins CD pipeline (written in Jenkinsfile).

- Report via **post** section; failing→**UNSTABLE**.

Team Dynamics =the psychological forces that influence the direction of a team's performance.

Effective team=right tools+right people+right practices

Tuckman Development Model (phases of group development): 1. **FORMING**: High degree of guidance needed from manager,individual roles unclear, process not well established

2. **STORMING**: Understanding how team decisions are made, purpose is clear but team relationships are blurry
3. **NORMING**: well-understood team relationships, commitment to team goals, begin to optimize process
4. **PERFORMING**: Team is performing well, focus is on being strategic, running well with little oversight

Waterfall vs. Agile team:

Traditional teams rely on hierarchical structure & specialized roles; Agile teams emphasize collaboration, self-organization & continuous adaptation.

High-Speeded Agile Teams (Self-organizing)
Cross-functional; Include engineers, designers, sales
Mutual respect and mutual responsibility: No blaming avoid "throw it over the wall" mentality
Sound engineering practices: Use appropriate tools.
Values of agile practices: Team members understand and embrace agile methodology.
Apply agile practices effectively: Both as individuals and as a team. (not good if only as individual)
Receive continuous training: Technical and non-technical training with team monitoring/coaching

Agile Manifesto (critical: a tech lead for 4-5 ppl)

- Individuals and interactions > processes and tools
- Working software(primary) > compreh documentation
- Customer collaboration > contract negotiation
- Responding to change > following a plan

Agile principles (blue→plan & estimate)

1. Highest priority: satisfy the customer→early & continuous delivery of valuable software.
2. Welcome changing requirements, even late in development. improve customer advantage.
3. Deliver working software frequently, preferring shorter timescale. rapid feedback.
4. Business & devs work together daily. Ensure close collaboration for shared understanding.
5. Build projects around motivated individuals; give them support and trust. empower teams.
6. Face-to-face conversation is the most effective communication method. clarity/speed.
7. Sustainable development at a constant pace. Maintain steady productivity without burnout.
8. Continuous attention to technical excellence and good design. allows easy adaptation.
9. Simplicity is essential. avoid unnecssy work.
10. Regularly reflect effective & adjust accordingly.

Scrum Theory: Empirical process control

- **Transparency**: significant process aspects must be visible to those responsible for the outcome
- **Inspection**: Scrum users must frequently inspect Scrum artifacts and progress toward an iteration goal to detect undesirable variances
- **Adaptation**: adjust the process aspects that lead to deviation outside the acceptable limits

Scrum Values-Commitment: Team members commit to achieving Team goals and supporting each other.
Courage: Members can do the right thing and work on tough problems. **Focus**: Everyone focuses on the Sprint work and Scrum Team goals. **Openness**: Team and the stakeholders are transparent about work and challenges. **Respect**: Team members respect each other to be capable and independent.

Roles: Feature: Small enough to be agile, Self-O, Cross-F, Deliver products iteratively & incrementally, maximizing opportunities for feedback. **Product owner**: Manage Product Backlog→**clear, concise, prioritised** (value to the customer) and transparent. (only PO). **Dev team**: optimal size 4-9, 1 lead w 4-5p. **Scrum master**: Keep team focus Scrum, facilitates events, removes impediments for PO and dev.

Sprint (2-4 weeks, fixed length):

1. sprint planning (2-part meeting)

- part1(max.4h): PO & dev select user stories
 - part2(max.4h): dev & PO break story into tasks
 - Output: Sprint Backlog (selected user stories)
2. **Daily Scrum meeting** (15mins inside team.):
 3. **Sprint Review** (max.4h. update with client):
 4. **Retrospectives** (max. 1-2h. inside team)

Requirements Elicitation

- Functional (details of input and output)
- Non-functional (performance/security/usability)

Requirements Documentation check:

Validity; Consistency; Completeness; Feasibility
BDD (behavior drive development): 1st write behavior (business value→acceptance criteria(test)→code) using **Given/When/Then** canvas to Improves communics among users, testers, devs.

Epic: big stories get split into smaller US across sprints

User Story: As a [stakeholder], I want a [feature], so that [benefit]. SMART US: Specific, Measurable (acceptance criteria), Achievable (fits in 1 Sprint; subdivide), Relevant (5 Whys→uncover business value), Timeboxed (estimate;stop if allocated time over)

Acceptance Criteria & "Done": Conditions of satisfaction: basis for **acceptance tests**.

Explicit = written user stories (explicitly specified as user stories developed by stakeholders in BDD) → acceptance tests (client need)

Implicit = "obvious" expectations (not specified in the user stories, professional duty)

→ integration tests (interactions between modules)

Imperative = **lots of UI detail**, complicated WHEN statements, lots of AND steps. Scenarios should rather focus on the application's behaviour

Declarative = **domain-language focus on behaviour** what business outcome/behavior should happen, in the domain language (e.g., "award an achievement").

Agile planning (Separate size from duration. First estimate size, then convert to time.US=units of estims)

Story point: # of story points~the overall size of story story point scale: 1,2,3,5(1.6x) / 1,2,4,8(2x). Why scale in Fibonacci/power of two? **reflect uncertainty; encourage split larger US; use small to predict large.**

Ideal days: "pure work" days without interruptions. (elapsed days=with interruptions).

Re-estimating only when your opinion of the relative size of 1+ stories has changed.

Story Points vs. Ideal Days. SP: help drive cross-functional behavior; Pure measure of size independent of time; Estimating in SP is faster; My IDs aren't your IDs. ID: easier to explain outside the team; easier to estimate at first; make velocity predictions easier

Techniques in estimation: Expert opinion (estimates based on opinion). Analogy: compare with other stories
Disaggregation: splitting a story into smaller tasks.
Planning Poker: based on expert opinion, analogy, disaggregation and fun. **Main benefit: avoids individual influence/bias by having members think independently.**

Velocity = SPs completed per sprint. use for forecasts.

Estimate # of Sprints ≈ total SP/velocity→weeks

Burndown = SPs left in this sprint backlog over time

Burndown chart = Tracks completion of development work throughout Sprint; (Forecast progress and show remaining). A gap in burndown chart and guideline→you may not finish all stories by the end of sprint.

Legal vs Ethical vs Moral? Morals: Principles that guide personal behavior→your personal compass. **Ethics**: shared rules of conducts for a social-context/community (e.g., ACM/IEEE codes).**Legal**: what the government enforces. Sometimes legal ≠ ethical (and vice-versa).

Intellectual Property (IP) "A category of property that includes intangible creations of the human intellect"

- **Copyright**: protects original expressions (source code, art)—not ideas/methods. Auto-granted in AU.
- **Trademarks**: protect names/logos that distinguish a product/service. Need registration.
- **Trade secrets**: protects formula/design/practice and provide a competitive advantage.
- **Patents**: protects idea/invention. public disclose idea; others can't use/sell for ~20 years. Criteria: new, non-obvious, useful, meet certain standard.

"Open-source" is opposite to "proprietary"

Open-source Software (OSS) vs. Free Software (FSF)
FSF stresses user freedom (the 4 freedoms: run, study/modify, redistribute original, distribute derived).

OSS stresses practical collaborative benefits.

Permissions for all licenses (except MIT not have 1)

1. Patent use (**can use the patent in the original work**)

2. Commercial
3. Distribution
4. Modification
5. Private use

Conditions

1. file-level CL
2. **library CL(GPL)**
3. **strong CL**
4. network CL

5. Disclose source **Strong CL→conditioned on making**

6. License and copyright notice(attribution)
7. State changes to **source code** **available complete source code of licensed works and modifications under the same license, include larger works using a licensed**
- Limitations** **work, under the same license)**

1. No liability (legal responsibility for harm)
2. No warranty (promise about quality/performance)
3. No permission to use the trademark in original work

Permissive (If no condition 1-4, is opposite to CopyLeft): derived work may distributed in different license and without providing source code.

Why separate network distribution? a private server hosting interact-able software on network-not count as distribution. GPL can't make it open-source, AGPL can.

Name	Conditions	Limitations
Apache 2.0	6, 7	1, 2, 3
MPL-2.0	1, 5, 6	1, 2, 3
LGPL	2, 5, 6, 7	1, 2
GPL-v3/v2	3, 5, 6, 7	1, 2
AGPL-3.0	3, 4, 5, 6, 7	1, 2